

BAB 13 – LOGIN, LOGOUT, DAN SESSION (Pertemuan 6)

Tujuan Pembelajaran

Setelah menyelesaikan bab ini, mahasiswa diharapkan mampu:

1. Membuat sistem **login dan logout** sederhana menggunakan CodeIgniter 4.
2. Menggunakan **session** untuk menyimpan status pengguna yang sedang login.
3. Membatasi akses halaman agar hanya pengguna yang login yang dapat masuk (proteksi halaman).
4. Menampilkan pesan flashdata untuk notifikasi login berhasil atau gagal.

Konsep Dasar Autentikasi

Autentikasi adalah proses memverifikasi identitas pengguna sebelum memberi akses ke sistem.

Dalam CodeIgniter 4, kita bisa menggunakan:

- **Session** → untuk menyimpan status login (misalnya `isLoggedIn = true`).
- **Helper redirect() dan Flashdata** → untuk menavigasi dan menampilkan pesan.

Alur login dasar:

1. User mengisi form login (username & password).
2. Sistem memverifikasi ke tabel `users`.
3. Jika benar → buat session dan arahkan ke halaman dashboard.
4. Jika salah → tampilkan pesan error.

Langkah-Langkah Praktikum

1. Membuat Tabel User

Masuk ke phpMyAdmin, lalu buat tabel user:

```
CREATE TABLE users (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  username VARCHAR(50) NOT NULL UNIQUE,  
  password VARCHAR(255) NOT NULL,  
  nama VARCHAR(100),  
  role ENUM('admin','user') DEFAULT 'user',
```

```
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Tambahkan satu akun admin:

```
INSERT INTO users (username, password, nama, role)
VALUES ('admin', PASSWORD('12345'), 'Administrator', 'admin');
```

 Catatan:

Jika MySQL Anda tidak mendukung fungsi `PASSWORD()`, gunakan enkripsi manual nanti lewat PHP menggunakan `password_hash()`.

2. Membuat Model User

Buat file:

```
app/Models/UserModel.php
```

Isi dengan kode:

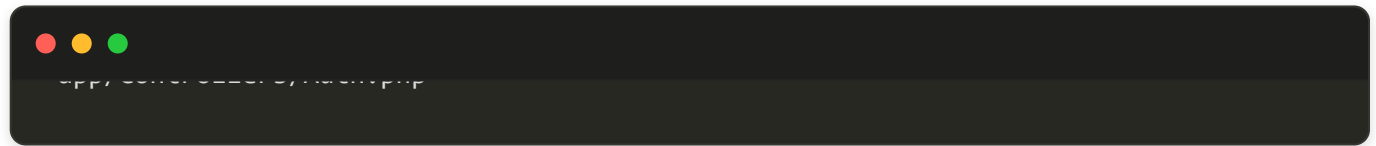
```
<?php

namespace App\Models;
use CodeIgniter\Model;

class UserModel extends Model
{
    protected $table = 'users';
    protected $primaryKey = 'id';
    protected $allowedFields = ['username', 'password', 'nama', 'role'];
}
```

3. Membuat Controller Auth

Buat file:



Isi dengan:



```

        return redirect()->to('/dashboard')->with('success', 'Selamat
datang, ' . $user['nama']);
    } else {
        return redirect()->back()->with('error', 'Password salah!');
    }
} else {
    return redirect()->back()->with('error', 'Username tidak ditemukan!');
}
}

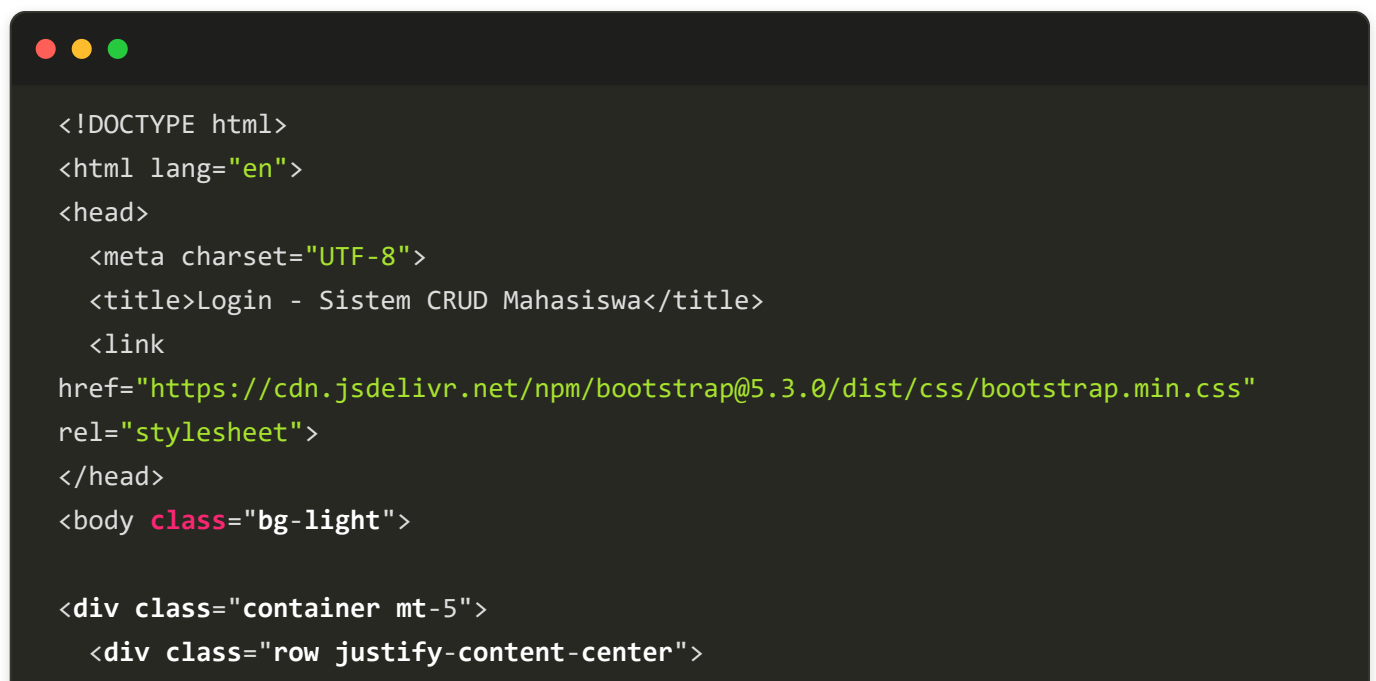
public function logout()
{
    session()->destroy();
    return redirect()->to('/login')->with('success', 'Anda telah logout.');
```

4. Membuat Halaman Login

Buat folder dan file:



Isi dengan:



```

<div class="col-md-4">
  <div class="card shadow-sm">
    <div class="card-header text-center bg-primary text-white">
      <h4>Login</h4>
    </div>
    <div class="card-body">

      <?php if (session()->getFlashdata('error')): ?>
        <div class="alert alert-danger"><?= session()->getFlashdata('error') ?
></div>
      <?php endif; ?>

      <?php if (session()->getFlashdata('success')): ?>
        <div class="alert alert-success"><?= session()-
>getFlashdata('success') ?></div>
      <?php endif; ?>

      <form action="/auth/processLogin" method="post">
        <div class="mb-3">
          <label>Username</label>
          <input type="text" name="username" class="form-control" required>
        </div>
        <div class="mb-3">
          <label>Password</label>
          <input type="password" name="password" class="form-control"
required>
        </div>
        <button class="btn btn-primary w-100">Login</button>
      </form>

    </div>
  </div>
</div>
</div>
</div>
</div>
</body>
</html>

```

5. Menambahkan Routing

Buka:



```
app/Controllers/AuthController.php
```

Tambahkan:



```
$routes->get('/login', 'Auth::login');
$routes->post('/auth/processLogin', 'Auth::processLogin');
$routes->get('/logout', 'Auth::logout');
```

6. Membuat Middleware (Filter) untuk Proteksi Halaman

Buat file baru:



```
app/Filter/AuthFilter.php
```

Isi dengan:



```
<?php

namespace App\Filters;
use CodeIgniter\HTTP\RequestInterface;
use CodeIgniter\HTTP\ResponseInterface;
use CodeIgniter\Filters\FilterInterface;

class AuthFilter implements FilterInterface
{
    public function before(RequestInterface $request, $arguments = null)
    {
        if (!session()->get('isLoggedIn')) {
            return redirect()->to('/login')->with('error', 'Silakan login terlebih dahulu!');
        }
    }

    public function after(RequestInterface $request, ResponseInterface $response,
```

```
$arguments = null)
{
    // Tidak ada aksi setelah
}
}
```

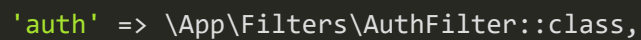
7. Daftarkan Filter

Buka file:



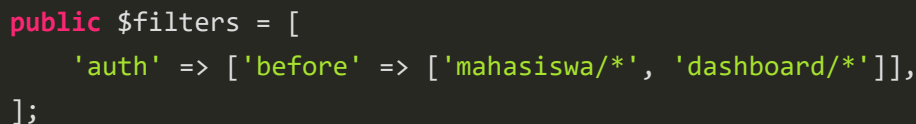
```
app/Controllers/AuthController.php
```

Tambahkan di `$aliases` :



```
'auth' => \App\Filters\AuthFilter::class,
```

Kemudian di `$globals` atau `$filters` tambahkan rute yang ingin diproteksi:



```
public $filters = [
    'auth' => ['before' => ['mahasiswa/*', 'dashboard/*']],
];
```

Dengan demikian, halaman `mahasiswa` hanya dapat diakses setelah login.

8. Membuat Controller Dashboard

Buat file:



```
app/Controllers/DashboardController.php
```

Isi dengan:

```
<?php

namespace App\Controllers;

class Dashboard extends BaseController
{
    public function index()
    {
        return view('dashboard/index');
    }
}
```

9. Membuat View Dashboard

Buat file:

```
app/Views/dashboard/index.php
```

Isi dengan:

```
<?= $this->include('layouts/header') ?>

<div class="d-flex justify-content-between align-items-center mb-3">
    <h3>Selamat Datang, <?= session('nama') ?>!</h3>
    <a href="/logout" class="btn btn-danger">Logout</a>
</div>

<p>Anda telah login sebagai <strong><?= session('role') ?></strong>.</p>

<hr>
<a href="/mahasiswa" class="btn btn-primary">Kelola Data Mahasiswa</a>

<?= $this->include('layouts/footer') ?>
```


❄ Uji Coba

1. Jalankan server:

```
php spark serve
```

2. Akses:

👉 `http://localhost:8080/login`

3. Login dengan:

- Username: `admin`
- Password: `12345`

4. Jika berhasil login, pengguna diarahkan ke dashboard.

5. Coba akses `/mahasiswa` tanpa login — sistem harus redirect ke halaman login.

💬 Latihan Mahasiswa

1. Tambahkan fitur **registrasi user baru** (username, nama, password).
2. Tambahkan kolom **last_login** di tabel `users`, dan update nilainya setiap kali user login.
3. Buat halaman profil pengguna yang menampilkan nama dan role user dari session.

📄 Tugas Praktikum

Buat sistem login sederhana untuk halaman CRUD santri:

- Hanya user yang login sebagai **admin** dapat menambah, mengedit, dan menghapus data.
- Gunakan **role-based access** untuk memproteksi fitur tertentu.
- Simpan password menggunakan `password_hash()`.

💡 Tips Keamanan

- Jangan pernah menyimpan password dalam bentuk teks biasa (plain text).
- Gunakan `password_hash()` untuk menyimpan, dan `password_verify()` untuk memverifikasi.
- Gunakan filter (middleware) untuk melindungi semua halaman CRUD.
- Tambahkan **CSRF token** di form untuk mencegah serangan Cross Site Request Forgery.